

Factory

Game Design Document

Nathan Tebbs · v0.1.0 · 2026-06-27

Build automated resource chains in an isometric factory. Watch simple rules create emergent complexity as you scale from a single workshop to an infinite empire.

Contents

1. Overview	4
1.1. Concept Summary	4
1.2. Target Experience	4
1.3. Influences & Reference Games	4
1.4. Scope & Platform	4
2. Core Design Pillars	5
3. Gameplay	5
3.1. The Core Loop	5
3.1.1. Micro Loop (seconds)	5
3.1.2. Mid Loop (minutes)	5
3.1.3. Macro Loop (hours / sessions)	5
3.2. Tension & Conflict	6
3.3. Failure States	6
3.4. Player Agency	6
4. Systems	6
4.1. Resource Model	6
4.1.1. Resource Types	7
4.1.2. Production Chains	7
4.1.3. Sinks	7
4.2. Machines & Structures	7
4.2.1. Machine Design Rules	7
4.2.2. Tiers & Progression	8
4.3. Map & Space	8
4.3.1. Progression Through Space	8
4.4. Progression & Unlocks	8
4.4.1. Early Game (30 min – 2 hours)	9
4.4.2. Mid Game (2 – 10 hours)	9
4.4.3. Late Game / Endgame (10+ hours)	9
4.5. Economy & Balance	9
5. UI & UX	10
5.1. Visual Language	10
5.2. HUD & Overlays	10
5.3. Feedback & Juice	10
5.4. Controls & Input	11
6. Art Direction	11
6.1. Style & Constraints	11
6.2. Color System	11
6.3. Machine Visual Design	12
6.4. Animation & Motion	12
7. Audio	12
7.1. Music	12

7.2. Sound Effects	12
8. Technical Notes	13
8.1. Simulation Model	13
8.2. Save & Load	13
8.3. Mod / Data-Driven Support	13
9. Roadmap	14
9.1. Milestone 1 — Prototype (1–2 weeks)	14
9.2. Milestone 2 — Vertical Slice (2–3 weeks)	14
9.3. Milestone 3 — Alpha (4–6 weeks)	14
9.4. Milestone 4 — Beta / Release (2–4 weeks)	14
10. Open Questions	15

1. Overview

1.1. Concept Summary

Factory is an idle/active automation game in isometric 2D. The player builds machines and connects them to create resource-production chains. A single machine produces one item per tick; chains combine machines into increasingly complex pipelines. The player expands indefinitely, discovering new machine types and resource combinations as they grow.

The emotional fantasy: watching chaos resolve into order. Building one machine feels simple. Connecting five creates a small system. Twenty machines create emergent patterns — bottlenecks you didn't predict, clever routing solutions, the satisfaction of seeing a complex web run smoothly.

1.2. Target Experience

Clarity through scale. Early, the player is in control — they see every piece move. As they grow, they stop micromanaging individual machines and start thinking in systems. They should feel the shift from “I built this” to “I designed this and it runs itself.” There's no win condition; the game is a sandbox that rewards you for making interesting choices, not for reaching a number. A player who builds 50 elegant machines should feel just as successful as one who scales to 1000.

1.3. Influences & Reference Games

- **Factorio:** Resource chains, production bottlenecks, the joy of optimization. We're borrowing the core loop but scaling it down and making it isometric/visual.
- **Infinifactory:** Spatial puzzle solving, constraint-based design, the aesthetics of simple geometric machines.
- **Islanders:** Relaxing progression without pressure or loss. Build at your own pace.
- **Dwarf Fortress:** Complex systems emerging from simple rules. We're not going for simulation depth, but the philosophy that restriction breeds creativity.
- **Wookash (Anton Waern):** Code structure and entity design philosophy. Intrusive lists, fat structs, minimal abstraction overhead.

1.4. Scope & Platform

- **Platform:** PC (macOS, Linux, Windows via itch.io or self-hosted)
- **Engine/Stack:** Custom C engine (mach) + SDL3
- **Target dev time:** 3–6 months for a playable vertical slice, open-ended scaling after
- **Solo:** Solo developer
- **Art style constraint:** Isometric 2D, tile-based grid. No hand-drawn sprites. Visual vocabulary: axis-aligned rectangles and simple geometric shapes. Color encodes

machine type and state. All machines fit the same isometric footprint for easy composition.

2. Core Design Pillars

1. **Complexity from simplicity.** Each machine has one job. Chains emerge from connecting simple rules. Interesting problems arise not from special cases but from putting basic pieces together.
2. **The player is always in control.** No randomness, no surprises, no blackbox AI. Every resource flow is visible. A mistake is always the player's choice, never the game's whim.
3. **Progress is visible, not gated.** There's no tech tree gatekeeping. If a machine exists, the player can build it from the start (cost is the only gate). Growth comes from the player's ambition, not arbitrary unlock lists.
4. **Scale is graceful.** Ten machines should play as smoothly as a hundred. The game doesn't become unmanageable, just richer. Camera zoom and smart rendering keep the interface clean even at large scales.

3. Gameplay

3.1. The Core Loop

The player places machines on a grid, connects them with belts/pipes, and watches resources flow. As production grows, they optimize bottlenecks. New machines unlock new possibilities, which invite new chains.

3.1.1. Micro Loop (seconds)

Click to place a machine → see it appear on the grid → watch it begin to work. Place a connection (belt) → resources start flowing through it. These actions have immediate visual feedback and feel snappy.

3.1.2. Mid Loop (minutes)

The player is solving a production problem: "I need more iron to build my factories." They might extend a mining chain, reroute an existing line, or design a new production path. The feedback loop is tight — they see the impact immediately in the flow of resources.

3.1.3. Macro Loop (hours / sessions)

The player expands their territory, discovers new machine types (either through gameplay or by exploring), and experiments with new chains. Early sessions focus on simple loops (mine → smelt → store). Later sessions add complexity (chains that feed

each other, loops within loops). Sessions can be short (30 min) or long (hours of idle gameplay in the background).

3.2. Tension & Conflict

Space and scarcity. The map is finite (initially). Machines take up space. Expansion costs resources. The player must choose: optimize existing chains to fit more, or spend resources to expand the map. As they scale, they encounter resource bottlenecks — not all resources are equally available, and some chains are longer than others. The tension is self-imposed: the player sets the goal (“I want 1000 iron per minute”) and must solve the puzzle of how to achieve it with limited space and resources.

3.3. Failure States

There is no failure or loss condition. The game is a sandbox — you can’t break it. “Doing poorly” means building an inefficient chain that doesn’t accomplish what you wanted. The player cares because they want their factory to **feel good** — elegant, well-organized, satisfying to watch. A factory that works is a victory. A factory that’s messy but still works is a problem waiting to be solved.

3.4. Player Agency

- **Machine placement:** Where to put a machine shapes the whole factory. Optimize for compact clusters or separate concerns spatially? This affects throughput and readability.
- **Connection routing:** How to connect machines — direct path, shared corridors, redundant routes? Changes the visual complexity and resilience of the system.
- **Scaling strategy:** Expand the map early (expensive, opens space) or optimize existing space (cheap, harder). Both are valid.
- **Production targets:** The player sets their own goals. Build toward a specific rate, or focus on complexity and exploration? Different playstyles are equally valid.
- **Resource specialization:** Early, the player produces iron, copper, wood. Later, they decide which resources to focus on and which chains to build. Specialization creates dependency and interesting routing puzzles.

4. Systems

4.1. Resource Model

Raw resources (ore, wood, stone) are mined from the environment. Machines transform them (ore → metal, wood → planks). Advanced machines combine resources into products (metal + wood → structure). The loop: mine → refine → combine → store (or use for upgrades).

4.1.1. Resource Types

- **Ore** (iron, copper, tin): Mined from deposits. Converted to metal via smelter.
- **Stone**: Mined, used for construction and structures.
- **Wood**: Harvested from trees (renewable). Used for structures and fuel.
- **Metal** (iron, copper, etc.): Refined ore. Building block for most machines.
- **Fuel**: Wood or coal. Powers machines that need energy.
- **Structures** (gears, circuits, etc.): Refined materials. Used to build advanced machines.

4.1.2. Production Chains

- **Simple**: Mine iron → smelt to metal → store.
- **Medium**: Mine iron + copper → smelt each → combine into alloy → store.
- **Complex**: Alloy + wood → craft gears; gears + metal → craft machines; machines + fuel → run production line.

Depth: 3–5 tiers, enough to create interesting spatial puzzles without becoming overwhelming.

4.1.3. Sinks

- **Construction**: Building new machines consumes resources.
- **Map expansion**: Costs resources to unlock new territory.
- **Machine upgrades**: Speed, capacity, efficiency improvements.
- **Storage is not infinite**: Overflow loss or slowdown incentivizes building new chains.

4.2. Machines & Structures

Tier 1 (Early)

- **Miner**: Extracts ore/stone/wood from deposits. Output: 1 ore per tick.
- **Storage**: Holds resources. Capacity is limited; overflows are lost.
- **Belt**: Transports 1 item per tick.

Tier 2 (Mid)

- **Smelter**: Takes ore, outputs metal. 1 ore → 1 metal per tick.
- **Crafter**: Takes 2 inputs, combines them. 1 ore + 1 wood → 1 structure per tick.
- **Splitter**: Takes 1 input, sends to 2 outputs (round-robin).

Tier 3 (Late)

- **Advanced crafter**: Takes 3 inputs, creates specialized products.
- **Furnace**: High-capacity smelter, slower but higher throughput.
- **Distributor**: Sends items to multiple outputs with priority control.

4.2.1. Machine Design Rules

- **Grid-based placement**. Each machine occupies one isometric tile.
- **Belts connect machines**. Items flow one per tick. Belts have implicit queues (one item per belt).

- **Machines have input/output ports.** A belt connects to one input and one output port per machine.
- **No machine capacity limits** — only storage has limits. Machines process whatever arrives.
- **All machines tick at the same rate** — synchronized. No complex scheduling needed.

4.2.2. Tiers & Progression

Machines are not locked by tech trees. All machines are available from the start. What gates them: cost and discovery. Early, the player has access to basic machines. As they build and explore, they discover (or unlock through expansion) advanced variants. A higher-tier machine is not “strictly better” — it’s **different**. A furnace is slower than a smelter but holds more ore, enabling different strategies.

4.3. Map & Space

Isometric grid. The world is a grid of isometric tiles. The player starts with a small region (e.g., 16×16 tiles). Space is finite and valuable — a core constraint that shapes decisions.

Resource deposits. Deposits of ore, stone, and wood are scattered across the map. Not uniformly distributed; some areas are resource-rich, others are sparse. This encourages exploration and strategic expansion.

Ore scarcity. Iron deposits run out after mining a certain amount (e.g., 1000 ore per deposit). This creates pressure to expand and find new deposits. Without scarcity, players could theoretically stay in one spot forever.

4.3.1. Progression Through Space

Early: The player stays in one small area, mining and processing what’s nearby.

Mid: They expand the map boundary to access new deposits. Expansion costs resources (e.g., 100 wood to unlock a new 8×8 region).

Late: The player has multiple regions, each specialized (one for iron, one for copper). Connections between regions create interesting routing puzzles.

Endgame: The map is procedurally infinite. As the player reaches the edge, new territory is generated with new deposits (or previously exhausted ore is replaced). The game has no hard boundary.

4.4. Progression & Unlocks

No tech tree. All machines are available, but knowledge is gated. Early machines are simple; advanced machines are discovered by exploring, building, and experimentation. Progression is about **enabling new strategies**, not unlocking power.

4.4.1. Early Game (30 min – 2 hours)

The player has: miner, storage, belt, basic smelter.

Goal: Set up a simple iron-mining and smelting loop. Experience: learning how belts move items, how timing works, how storage fills up.

Challenge: Limited space and ore. Forces the player to be intentional about placement.

4.4.2. Mid Game (2 – 10 hours)

The player has expanded the map and discovered crafter machines. They now combine resources into new products (metal + wood → structures). Multiple chains run in parallel.

Goal: Build a more complex factory that feeds itself. Set production targets (e.g., “1000 structures per minute”).

Challenge: Space is still tight. Optimization becomes necessary. Routing becomes a puzzle.

4.4.3. Late Game / Endgame (10+ hours)

The map is large or procedurally infinite. The player experiments with massive scales (1000+ machines). They optimize for aesthetic or efficiency. They invent new designs.

Endgame mechanics: Optional challenges (“achieve 10k ore/min”), or simply the satisfaction of a beautiful, functioning system. No prestige loop — the game doesn’t reset. Players can keep building indefinitely.

4.5. Economy & Balance

Production speed: Early, a single miner produces 1 ore/tick. A smelter processes 1 ore/tick. A belt moves 1 item/tick. This keeps early gameplay legible — the player can see individual items moving. No overwhelming throughput. As the player scales, they group items together (visual bundles) to avoid slowdown.

Resource costs: Building a miner costs 50 wood. Building a smelter costs 50 metal. This creates a bootstrapping puzzle — you need some initial resources to build machines, which produce more resources. Cost scaling prevents players from trivializing the game by spamming cheap machines.

Ore depletion: Each deposit holds 1000 ore. At 1 ore/tick, a single miner depletes a deposit in 17 minutes (real time). This incentivizes expansion before current deposits run out.

Map expansion: Unlocking a new 8×8 region costs 100 wood or equivalent. Expensive enough to matter, cheap enough to feel achievable within a few minutes of active play.

No dominant strategy: Different machine combinations are equally valid. A player can optimize for speed (lots of belts, high throughput) or space efficiency (compact clusters).

Both reach the same production targets; the trade-off is in aesthetics and complexity management.

5. UI & UX

5.1. Visual Language

Isometric perspective. 45-degree angle view shows depth and spatial relationships. Machines are geometric shapes (rectangles, cylinders) rendered in isometric projection. No hand-drawn details; all shapes are procedurally rendered.

Color encodes type and state. Machine color indicates its function (miners are brown, smelters are orange, storage is gray). Belt color indicates the resource type (iron is red, copper is orange, wood is brown). Machine outline brightness indicates state (bright = active, dim = idle, red = blocked/overfull).

Size and hierarchy. Important machines (miners, furnaces) are larger. Support infrastructure (belts) is thinner. Storage is visually distinct (cube-like). This lets the player scan a large factory and immediately understand the structure.

5.2. HUD & Overlays

Top-left: Current resources (wood: 250, ore: 150, metal: 50). Simple text, minimal visual noise.

Top-right: Tool palette. Selected tool is highlighted. Mouse hover shows cost.

Bottom: Hotkeys. “Q” for belt mode, “E” for eraser, “1” for miner, etc. Only shown during early game; hidden after tutorial.

When placing: Ghost preview of what you’re about to build. Green if placeable, red if blocked.

Click on a machine: Details popup showing input/output rates, current queue length, uptime. Dismissible.

5.3. Feedback & Juice

- **Machine placed:** Brief “pop” animation and sound. Satisfying but not distracting.
- **Resource delivered:** Item visibly travels along belt, drops into destination. Clear cause-and-effect.
- **Bottleneck cleared:** Visual change (color, brightness) indicates machine unblocked. No fanfare, just clarity.
- **Map expanded:** New territory fades in. Satisfying reveal of new space and deposits.
- **Storage full:** Visual alert (red outline) and gentle sound. Doesn’t interrupt; just draws attention.

5.4. Controls & Input

Mouse-driven primary. Click to place machines, click to connect belts. Right-click or Escape to cancel. Scroll wheel to zoom. Pan with middle-click or arrow keys.

Keyboard shortcuts for power users:

- Q: Belt mode
- E: Eraser
- 1–9: Quick-select machines
- Space: Pause/unpause simulation
- R: Rotate (for machines with directional input/output)

First 60 seconds: “Click to place a miner” tooltip. After first machine, the interface is self-explanatory through UI hints.

6. Art Direction

6.1. Style & Constraints

Isometric 2D, procedural rendering. All machines and tiles are drawn as geometric shapes (rectangles, cylinders, etc.) in isometric projection. No pre-drawn sprites. Rendering is done in code via SDL3 primitives (filled quads, lines, circles).

One tile size: All machines fit a standard isometric footprint. Makes composition simple and grid-based. Avoids asset sprawl.

Programmer art. Simple color blocks with outlines. Clarity over realism. A factory should be readable at a glance.

Out of scope: Hand-drawn details, smooth curves, complex lighting/shadows, particle effects (initially). These can be added later; core gameplay works without them.

6.2. Color System

Resource type: Iron (red), copper (orange), wood (brown), stone (gray), generic output (white).

Machine function:

- Miners: brown/tan
- Smelters: orange
- Crafters: purple
- Storage: gray
- Belts: color of resource

State (outline/brightness):

- Active: bright, normal outline
- Idle: dim, thin outline
- Blocked: red outline, pulsing
- Overfull: red tint, thick outline

This grammar lets the player understand a complex factory without detailed inspection.

6.3. Machine Visual Design

All machines: Simple isometric rectangle with a border. Internal elements (glyph or icon) indicate function.

Miners: Pickaxe glyph in the center.

Smelters: Flame or pot glyph.

Crafters: Gear or hammer glyph.

Storage: Larger cube, divided into grid sections to suggest capacity.

Belts: Thin line with animated flow direction.

Different glyphs are instantly recognizable even in dense factories.

6.4. Animation & Motion

- **Belts:** Items slide along belts smoothly. Arrow indicators show flow direction (animated dashes).
- **Machine activation:** Glyph pulses or glows when active.
- **Item arrival:** Brief scale-up/pop when item reaches destination.
- **Map reveal:** New territory fades in over 0.5 seconds.
- **UI interactions:** Button clicks have subtle scale feedback. Palette selection highlights.

Minimal, clear, satisfying. No lag or unnecessary motion.

7. Audio

Audio is secondary but important for juice and feedback.

7.1. Music

Ambient, loop-able track. Calm, rhythmic, industrial-ish vibe. Think minimal synth or lo-fi ambient. The music should not compete with the sound design; it sits in the background and sets the mood. One track is enough for full playability; variations can be added later.

7.2. Sound Effects

Critical:

- Machine placed: Short percussive hit (0.1s). Satisfying but not jarring.
- Belt activated: Soft mechanical whine (looping, very quiet). Indicates machines are running.
- Item delivered: Small chime or click (0.05s). Feedback that the system is working.
- Storage full: Alarm tone (0.2s, once). Draws attention to problem.

Nice-to-have:

- Smelter active: Low rumble or hum (looping, quiet).
- Machine breaking: Grinding sound or thud (0.3s, once).
- Map expansion: Satisfying “woosh” or reveal sound (0.5s).

All SFX are subtle and loop-able. Nothing irritating on repeat. Can mute with toggle.

8. Technical Notes

8.1. Simulation Model

Tick-based synchronous simulation. Every machine ticks at the same rate (e.g., 60 ticks per second, or decoupled from render). Each tick, miners produce ore, belts move items, crafters process inputs. This keeps the game deterministic and predictable.

Decoupled sim and render. Simulation ticks at a fixed rate (e.g., 60Hz). Rendering can run faster and interpolates between sim states. This allows smooth camera panning and belt animations even at high sim rates.

Target scale: 1000+ machines should run smoothly on a modern CPU. Early optimization focuses on efficient entity iteration (fat structs, intrusive lists per entity type, minimal allocation).

8.2. Save & Load

Unit of save: Full world state. All machines, belts, resource quantities, map expansion level, player position/zoom.

Format: Binary or simple text (JSON-ish). Must be human-editable for debugging/modding.

Persistence: Auto-save every 60 seconds. Manual save on demand. One slot, no branching save states (simple design, less code).

8.3. Mod / Data-Driven Support

Initial: Machines and resources are hardcoded. Sim rules are fixed.

Future: JSON or Lua configuration for machine properties (speed, cost, input/output). This enables mods and easy rebalancing without recompiling.

No scripting initially. Keep gameplay logic in C. Lua can be added later for machine behaviors if needed.

9. Roadmap

9.1. Milestone 1 — Prototype (1–2 weeks)

Deliverable: Click to place miners and storage; watch ore appear in storage.

What you can play: Place a miner on ore deposit. It slowly fills storage. That's it. No belts, no crafters, no UI. Proves the core loop: place machine → watch it work.

Engine features demonstrated: Basic entity system, rendering isometric sprites, input handling, simple simulation loop.

Questions answered: Does isometric rendering work? Is the entity system fast enough? Does the core feedback loop feel good?

9.2. Milestone 2 — Vertical Slice (2–3 weeks)

Deliverable: Place miners, belts, smelters. Create a simple production chain: mine ore → smelt metal → store.

What you can play: A 10-minute session where you build and optimize a small factory. Early game in miniature.

Engine features demonstrated: Entity routing (belts), multi-machine interaction, storage/queue mechanics, UI for machine selection.

Questions answered: Does the belt/routing system feel good? Is the progression from simple to complex believable? Do players understand what to do intuitively?

9.3. Milestone 3 — Alpha (4–6 weeks)

Deliverable: All core machines, resource types, basic map expansion. Early/mid game complete.

What you can play: 2–5 hours of gameplay. Mine → smelt → craft. Expand the map. Optimize chains. First real challenge of scale and space.

Engine features demonstrated: Full entity system with all machine types, map tiling and expansion, save/load, balanced economy.

Questions answered: Does the game scale smoothly? Are there interesting decisions to make? Are bottlenecks fun to solve?

9.4. Milestone 4 — Beta / Release (2–4 weeks)

Deliverable: Polished, balanced, ready to ship.

What's added: Sound design, visual polish, performance optimization, balance tuning, edge-case handling.

Platform: Windows/macOS/Linux via itch.io. Self-hosted or Open Source (undecided).

Release checklist: Play through full progression, verify save/load, confirm no crashes, add help text, write minimal docs.

10. Open Questions

Decisions to make as development progresses:

- **Ore depletion mechanics:** Do deposits regenerate after time? How fast? Or is scarcity permanent, forcing constant expansion?
- **Belts as queues:** Can a belt hold multiple items, or just one? One-item belts are simpler but may feel bottlenecked. Queuing adds complexity.
- **Machine rotation:** Do miners/crafters have input/output ports on specific sides? Or are they omni-directional? Rotation adds puzzles; omni-directional is simpler.
- **Map boundaries:** Hard boundary (procedurally generated beyond edge) or truly infinite (generated on-demand)? Affects streaming/memory.
- **Resource overflow:** What happens when storage is full? Drop resources, pause belts, or burst release? Different feels and strategies.
- **Pause mechanic:** Can the player pause the simulation? Good for planning but removes time pressure. Include it?
- **Undo/redo:** Too powerful? Or necessary for a non-punishing sandbox? Defer to post-release.
- **Multiplayer:** Not in scope initially. Could be added via async co-op or shared world later.